



Apex Institute of Technology

Computer Science & Engineering

Experiment 9

Name: Trimann Kaur

UID: 24BAI70511

Branch: B.E. CSE (AIML)

Section: 24AIT_KRG-G1

Semester: 4

Date of Performance: 01.04.2026

Subject Name: Database
Management System

Subject Code: 24CSH-298

AIM: To design and implement stored procedures in PostgreSQL for performing Create, Read, Update, and Delete (CRUD) operations on database tables in an efficient and reusable manner.

OBJECTIVES:

- To understand the concept and importance of stored procedures.
- To implement parameterized stored procedures.
- To perform INSERT, UPDATE, DELETE, and SEARCH operations.
- To improve database performance and security.
- To gain industry-relevant procedural SQL experience.

SOFTWARE REQUIREMENTS:

- Database Management System:
 - PostgreSQL Database
- Database Administration Tool / Client Tool:
 - pgAdmin

PRACTICAL/EXPERIMENT STEPS:

1. A students table was created to store student details like ID, name, age, and course.
2. A stored procedure add_student was implemented to insert new records.
3. A function get_students was created to retrieve all records from the table.
4. A stored procedure update_student was designed to update existing student data.
5. A stored procedure delete_student was implemented to remove records using ID.
6. A function search_student was created to search students by name.
7. All CRUD operations were tested using procedure calls and SELECT statements.



Apex Institute of Technology

Computer Science & Engineering

PROCEDURE:

1. PostgreSQL (pgAdmin) was opened and the required database was selected.
2. The students table was created using CREATE TABLE with appropriate fields.
3. Stored procedures were written using CREATE OR REPLACE PROCEDURE for INSERT, UPDATE, and DELETE operations.
4. Functions were created using CREATE OR REPLACE FUNCTION for SELECT and SEARCH operations.
5. Parameters were passed to procedures to perform dynamic operations.
6. Procedures were executed using CALL and functions using SELECT.
7. The output was verified to ensure correct implementation of CRUD operations.

CODE:

```
CREATE TABLE students (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100),  
    age INT,  
    course VARCHAR(100)  
);  
  
CREATE OR REPLACE PROCEDURE add_student(  
    p_name VARCHAR,  
    p_age INT,  
    p_course VARCHAR  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    INSERT INTO students(name, age, course)  
    VALUES (p_name, p_age, p_course);  
END;  
$$;  
  
CALL add_student('Max', 20, 'BE');  
SELECT * FROM students  
  
CREATE OR REPLACE FUNCTION get_students()  
RETURNS TABLE(  
    id INT,  
    name VARCHAR,  
    age INT,  
    course VARCHAR
```



Apex Institute of Technology

Computer Science & Engineering

```
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    RETURN QUERY SELECT * FROM students;  
END;  
$$;  
  
SELECT * FROM get_students();  
  
CREATE OR REPLACE PROCEDURE update_student(  
    p_id INT,  
    p_name VARCHAR,  
    p_age INT,  
    p_course VARCHAR  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    UPDATE students  
    SET name = p_name,  
        age = p_age,  
        course = p_course  
    WHERE id = p_id;  
END;  
$$;  
  
CALL update_student(1, 'Max', 22, 'MBA');  
SELECT * FROM students  
  
CREATE OR REPLACE PROCEDURE delete_student(  
    p_id INT  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    DELETE FROM students  
    WHERE id = p_id;  
END;  
$$;
```



Apex Institute of Technology

Computer Science & Engineering

```
CALL delete_student(1);  
SELECT * FROM students
```

```
CREATE OR REPLACE FUNCTION search_student(p_id INT)  
RETURNS TABLE(student_id INT, name VARCHAR, age INT, course VARCHAR)  
AS $$  
BEGIN  
    RETURN QUERY  
    SELECT s.id, s.name, s.age, s.course  
    FROM students s  
    WHERE s.id = p_id;  
END;  
$$ LANGUAGE plpgsql;
```

```
SELECT * FROM search_student(1);
```

I/O ANALYSIS:

1. Create a table

The students table is successfully created with fields such as id, name, age, and course. The id is defined as a primary key with auto-increment, ensuring unique student records.

```
CREATE TABLE  
  
Query returned successfully in 387 msec.
```

2. Creating a Procedure (INSERT)

The stored procedure add_student is created successfully. It accepts parameters (name, age, course) and inserts a new record into the table. A new record is inserted into the students table successfully.

```
CREATE PROCEDURE  
  
Query returned successfully in 181 msec.  
  
CALL  
  
Query returned successfully in 111 msec.
```

3. Creating a Function (READ)

The function get_students is created to fetch all records from the table. Displays all student records present in the table.



Apex Institute of Technology

Computer Science & Engineering

	id [PK] integer	name character varying (100)	age integer	course character varying (100)
1	1	Max	20	BE

4. UPDATE Operation

The stored procedure `update_student` is created to update existing student details using ID. The record with `id = 1` is successfully updated with new values.

	id [PK] integer	name character varying (100)	age integer	course character varying (100)
1	1	Max	22	MBA

5. DELETE Operation

The stored procedure `delete_student` is created to delete a student record based on ID. The record with `id = 1` is removed from the table.

	id [PK] integer	name character varying (100)	age integer	course character varying (100)

6. SEARCH Operation

The function `search_student` is created to search records using `id`. It displays records with the given `id`.

	id [PK] integer	name character varying (100)	age integer	course character varying (100)

LEARNING OUTCOMES:

1. Gained understanding of stored procedures and functions in PostgreSQL for efficient database operations.
2. Learned to implement CRUD operations (Create, Read, Update, Delete) using parameterized procedures and functions.
3. Developed the ability to improve data handling, reusability, and security in database applications using PL/pgSQL.